问题 A: 熊二嘴馋了

时间限制: 1 Sec 内存限制: 128 MB

题目描述

熊二想吃到放在树顶上蜂蜜，假设树高 N 米，熊二白天爬高 X 米，晚上下滑 Y 米，请问，熊二需要几天才能吃到蜂蜜。

【输入】

输入数据为一行，三个整数 X，Y，N(1<=Y<X<=N<=100000000),其中，X 表示熊二白天爬高的距离，Y 表示熊二晚上下滑的距离，N 表示树的高度。

【输出】

输出一个整数，表示熊二需要几天才能吃到蜂蜜。

【输入输出样例 1】

snail.in	snail.out
2 1 5	4

【输入输出样例 2】

snail.in	snail.out
5 1 6	2

【数据范围】

30%的数据 N<=1000

100%的数据 N<=100000000

答案：

```
#include<iostream>

using namespace std;
typedef long long ll;
void solve()
{
    ll x,y,n;
    cin>>x>>y>>n;
    ll sum=0;
    for(int i=1;i<=n;i++)
    {
        sum+=x;
        if(sum>=n)
        {
            cout<<i<<endl;
            return;
        }
        sum-=y;
    }
}

int main()
{
    int t;
    solve();
    return 0;
}
```

问题 B: 横渡河流

时间限制: 3 Sec 内存限制: 128 MB

题目描述

M 个游泳爱好者来到水流湍急的河流，想要横渡到河对岸，但是只携带了一个浮力球，浮力球最多能保证两个人同时使用安全渡河。如果各自单独通过的话，M 个人所需要的时间已知：而如果两人同时通过河道，所需要的时间就是游得较慢的那个人单独行动时所需要的时间。问题是，如何设计一个方案，让这 M 个人尽快渡河。

输入

第一行是一个整数 T(1<=T<=20)表示测试数据的组数
每组测试数据的第一行为一个整数 M(1<=M<=1000)表示共有 M 个人要渡河
每组测试数据的第二行是 M 个整数 Xi,表示此人过河所需要花时间。(0<Xi<=100)

输出

输出所有人都渡过河流需要用的最少时间

样例输入 Copy

```
1

4

1 2 5 10
```

样例输出 Copy

```
17
```

对于一个规模比较大的问题，我们或许可以从小问题开始尝试来解决。
假设 N 个人过河所需要的时间保存在一个数组 nums 中，nums 经过排序由小到大，即过河时间由快到慢，假设时间分别为 abcdefg，a<b<c<d<e<f<g。
N=1 时，过河的时间为 a
N=2 时，过河所有的时间为 b
N=3 时，过河所需要的时间为 a+b+c(ab 先过去，所需时间为 b，a 再回来，所需时间为 a，然后 ac 在过去，所需时间为 c)
N=4 时，此时两种方案有:
方案 1：让 a 带最慢 d 先过去，然后 a 回来，再带次慢的 c 过去，a 再回来，最后 ab 一起过去。
此方案所需要的时间为 d+a+c+a+b=2a+b+c+d
方案 2：先让最快的两个人 ab 过去，然后 a 回来，再让最慢的两个人过去，b 回来，最后 ab 一起过去。此方案所需要的时间为 b+a+d+b+b=a+3b+d
具体采取哪种方案，只需要比较方案 1 与方案 2 哪个更小即可。

```
#include<iostream>
#include<algorithm>
using namespace std;
int a[1005];
int main(){
    int N;
    scanf("%d",&N);
    while(N--){
        int i,n,time=0;
        scanf("%d",&n);
        for(i=0;i<n;i++){
            scanf("%d",&a[i]);
        }
        sort(a,a+n);
        while(n>3){
            if(2*a[1]+a[0]+a[n-1]<a[n-2]+a[0]*2+a[n-1])
                time+=2*a[1]+a[0]+a[n-1];
            else
                time+=a[n-2]+a[0]*2+a[n-1];
             n-=2;
        }
        if(n==1)
            time+=a[0];
        if(n==2)
            time+=a[1];
        if(n==3)
            time+=a[0]+a[1]+a[2];
        printf("%d\n",time);
    }
    return 0;
}
```

```
import java.util.Arrays;
import java.util.Scanner;

/**
 * Created by wanglei on 19/4/13.
 */
public class CrossRiver {

    public static int time = 0;

    public static void main(String[] args) {
        Scanner in = new Scanner(System.in);
        int n = in.nextInt();
        int[] nums = new int[n];
        for (int j = 0; j < n; j++) {
            nums[j] = in.nextInt();
        }
        Arrays.sort(nums);
        cross(nums);
        System.out.println("time is: " + time);
```

```
    }

    // method1: ad 过去(d), a 回来(a), ac 过去(c), a 回来(a), ab 过去(b) all time: 2a+b+c+d
    // method2: ab 过去(b), a 回来(a), cd 过去(d), b 回来(b), ab 过去(b) all time: a+3b+d
    public static void cross(int[] nums) {
        if (nums.length == 1) {
            time += nums[0];
            return;
        }
        if (nums.length == 2) {
            time += nums[1];
            return;
        }
        if (nums.length == 3) {
            time += nums[0] + nums[1] + nums[2];
            return;
        }
        if (nums.length > 3) {
            if (nums[0] + nums[nums.length - 2] < 2 * nums[1]) {
                time += 2 * nums[0] + nums[nums.length - 2] + nums[nums.length -
1];
            } else {
                time += nums[0] + 2 * nums[1] + nums[nums.length - 1];
            }
            int[] tmp = Arrays.copyOfRange(nums, 0, nums.length - 2);
            cross(tmp);
        }
    }
}
```

问题 C: 斗罗大陆星斗大森林郊游

时间限制: 1 Sec 内存限制: 128 MB

题目描述

唐三和大师相约去星斗大森林，唐三先出发去找大明二明，速度为 A 公里每小时，T 小时后，大师带着罗三炮出发，以每小时 B 公里的速度去追唐三，而罗三炮则以

C 公里/小时的速度追赶唐三，追上唐三后立即返回，直到大师追上唐三。请问直到大师追上唐三时，罗三炮跑了多少公里？

输入

第一行要求输入一个整数 M(M<100)，表示数据的组数

每组数据占一行，是四个正整数，分别为 T,A,B,C（数据保证 A<B<C)

输出

输出罗三炮跑的路径，结果保留小数点后两位。

样例输入 Copy

```
1

5 10 15 20
```

样例输出 Copy

```
200.00
```

Mail the author

```java
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int M = scanner.nextInt();
        while (scanner.hasNext()) {

            int T = scanner.nextInt();
            int A = scanner.nextInt();
            int B = scanner.nextInt();
            int C = scanner.nextInt();


            double SanStart = A * T;
            double time1 ;
            double time2 ;

            time1 = SanStart / (C - A);
            double luo = time1 * C;
            time2 = SanStart / (B - A);
            double time3 = time2 - time1;

            double res = time3 * C + luo;

            System.out.println(String.format("%.2f",res));
        }
    }
}
/*****************************************************************
```

```
#include<iostream>
#include<algorithm>
#include<cstring>
#include<queue>
#include<vector>
#include<unordered_map>
#include<vector>
#include<string>
using namespace std;
typedef long long ll;
typedef pair<int,int> pii;
const int N=1000+10;
int n,m;

int ways[N];
int g[N][N];

int maxv;
int a[N];
void solve()
{
    double t,a,b,c;
    cin>>t>>a>>b>>c;
    b-=a;

    double res=t*a/b*c;

    string x=to_string(res);

    string k;
    int z=x.find('.');
    cout<<x.substr(0,z)<<'.'<<x.substr(z+1,2)<<endl;
}

int main()
{
    int t;
    cin>>t;

    while(t--) solve();

    return 0;
}
```

问题 D: 小浣熊干脆面

时间限制: 1 Sec 内存限制: 128 MB

题目描述

某商店中小浣熊干脆面为了促销，做了如下活动：三张水浒英雄卡可以换一袋小浣熊干脆面。Tom 攒了 10 张水浒英雄卡，他最多可以换多少袋小浣熊干脆面？答案是 5 袋，方法如下：先用 9 张水浒英雄卡换 3 袋小浣熊干脆面，吃掉 3 袋后，还有 4 张卡，再用 3

张卡换一袋干脆面，吃完后，还余下 2 张卡。然后再从商店处借出一袋干脆面，吃掉后，加上原有的 2 张卡，一共就拥有了 3 张卡，恰好用这 3 张卡换一袋干脆面还给商店。问如果 Tom 攒了 m 张水浒英雄卡，最多可以换多少袋小浣熊干脆面？

输入

输入数据最多为 10 组测试数据，存在一个输入文件中，一行一组数据，仅包含一个正整数 m（1<=*m*<=100），表示 Tom 收藏的水浒英雄卡的数量。*m*=0 表示输入结束，你的程序对这一行不做处理。

输出

对于每组测试数据，输出一行结果，表示最多可以吃的小浣熊干脆面的袋数。如果一袋也吃不到，输出为 0。

样例输入 Copy

```
3

10

81

0
```

样例输出 Copy

```
1

5

40
```

```java
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        while(sc.hasNext()){
            int n = sc.nextInt();
            if(n==0){
                break;
            }
            int count=0;

            while(n>=3){
                int find1=n/3;
                count+=find1;
                int yu1=n%3;
                n=yu1+find1;
            }
            if(n==2){
                count++;
            }
            System.out.println(count);
        }
    }
}

/*************************************************************
```

```cpp
#include<cstdio>
#include<algorithm>
#include<iostream>
using namespace std;



int main()
{
    int n;
    while(cin >> n)
    {
        if(n == 0)break;
        int b[1000] = {0};

        for(int i = 0;i < 1000;i ++)
        {
            b[i] = n / 3;
            n = n % 3 + b[i];

            if(n == 2)
            {
                b[++i] = 1;
                break;
            }
            else if(n < 2)
                break;
        }

        int sum = 0;
        for(int i = 0;i < 1000;i++)
            sum += b[i];

        cout << sum << endl;
    }
    return 0;
}
/*****************************************************
```

```python
def exchange(num):
    re=0
    while num//3:
        re+=num//3
        num=num%3+num//3
    if num==2:
        re+=1
    return re
while 1:
    num=int(input())
    if num<1 or num >100:
        break
    print(exchange(num))
```

问题 E: 光头强电锯分配

时间限制: 1 Sec 内存限制: 128 MB

题目描述

李老板拥有新型电锯 P 台，准备分给光头强等 Q 个伐木工。各伐木工若获得这些新型电锯用于日常伐木工作，可以大大提高伐木效率，为李老板提供更多的收入。问：如何分配这 P 台电锯才能使李老板得到的收入最高？求出最大收入值。其中 P <= 15，Q <= 10。分配原则：每个伐木工有权获得任意数目的电锯，但总数不超过电锯数 P。

输入

第 1 行包括两个值，第一个值是伐木工数 Q，第一个值是电锯数量 P。

接下来是一个 Q*P 的矩阵，表明了第 i 个伐木工分配 j 台电锯的收入。

输出

第 1 行输出最大收入值。

接下来 Q 行，每行 2 个数，即伐木工编号和该伐木工获得电锯台数。

样例输入 Copy

```
3 3
30 40 50
20 30 50
```

```
20 25 30
```

样例输出 Copy

```
70

1 1

2 1

3 1
```

按照伐木工的顺序来分配电锯，即按照伐木工的顺序划分阶段，第一个阶段把 P 台设备分给第一个伐木工，记录下来获得的各个盈利值，然后把 P 台设备分给前两个伐木工，和第一个阶段比较记录下来更优的各个盈利值，一直到第 Q 个阶段把 P 台电锯全部分给了 Q 个伐木工，就可以得到最优解，下面来讨论两个阶段之间的关系，设数组 f[i][j]表示前 i 个伐木工分配 j 台电锯的最大盈利，数组 f[i-1][k]表示前 i-1 个伐木工分配 K 台电锯的最大盈利（1<=i<=Q，1<=j<=P，0<=k<=j），用 value[i][j]表示第 i 个伐木工分配 j 台电锯的盈利，则 f[i][j]可以由下面的式子中取最大值获得：

f[i-1][0]+value[i][j]　　//前 i-1 伐木工分配 0 台电锯最大盈利＋第 1 个伐木工分配 j 台的电锯的盈利

f[i-1][1]+value[i][j-1]　　//前 i-1 伐木工分配 1 台电锯最大盈利＋第 1 个伐木工分配 j-1 台的电锯的盈利

f[i-1][2]+value[1][j-2]　　//前 i-1 伐木工分配 2 台电锯最大盈利＋第 1 个伐木工分配 j-2 台的电锯的盈利

f[i-1][j-1]+value[i][1]　　//前 i-1 伐木工 j-1 分配台电锯最大盈利＋第 1 个伐木工分配 1 台的电锯的盈利

f[i-1][j]＋value[i][0]　　//前 i-1 伐木工分配 j 台电锯最大盈利＋第 1 个伐木工分配 0 台的电锯的盈利

在这里用电锯数用做每个阶段的状态，由于 value[1][j]的值为定值，要使前面每个式子的值最大，就必须使第 i-1 阶段的各个状态的值最大，阶段 i 的状态只能由阶段 i-1 中的状态通过状态转移方程求得，与其他状态没有关系。由此可见，该问题具备了最优子结构和无后效性原则，适宜使用动态程序设计方法求解。

状态转移方程为：f[i][j]= max { f[i-1][K] + value[i][j-k] }　(1<=i<=Q,1<=j<=P, 0<=k<=j)
初始值：f[0][0]=0，f[n][m]的值即为所求最大盈利值。

```cpp
#include <iostream>
using namespace std;

int n, m;
int w[20][20], path[20], t = 0;
int res = 0, ans[20];

inline void dfs(int u, int v) // u 人 v 锯
{
    if(v == 0 || u == n + 1)
    {
        if(t > res)
        {
            res = t;
            for(int i = 1; i <= n; ++ i)
                ans[i] = path[i];
        }
        return ;
    }
    // 不选
    dfs(u + 1, v);
    // 选
    for(int i = 1; i <= v; ++ i)
    {
        t += w[u][i];
        path[u] = i;
        dfs(u + 1, v - i);
        path[u] = 0;
        t -= w[u][i];
    }
}

int main()
{
    scanf("%d%d", &n, &m); // n 人 m 锯
    for(int i = 1; i <= n; ++ i)
        for(int j = 1; j <= m; ++ j)
            scanf("%d", &w[i][j]);
    dfs(1, m);
    printf("%d\n", res);
    for(int i = 1; i <= n; ++ i)
    {
        printf("%d %d\n", i, ans[i]);
    }
    return 0;
}
```

问题 F: 小朋友游戏分组

时间限制: 1 Sec 内存限制: 128 MB

题目描述

有 m 个小朋友做游戏，每个人有一个编号，分别是 1,2，.....m,已知他们之间存在 t 个关系，其中关系是一个二元组（a,b)，表示 a,b 是同一组的组员。
当 m,t 和 t 个关系给出后，求出其中共有多少个组，人数最多的组有多少小朋友？
例如：m=6，t=3，三个关系为（1,2），(1,3)，(4,5)
此时，6 个人组成三组，即：｛1,2,3｝为一个组，｛4,5｝为一个组，｛6｝单独为一个组，第一个组的人数为最多，人数是 3 人。

输入

文件的第一行为 m,t 二个整数（1≤m≤100）（用空格分隔）
接下来的 t 行，每行二个整数（用空格分隔）表示关系

输出

二个整数（分别表示组个数和人数最多组人数）

样例输入 Copy

```
6 3
1 2
1 3
4 5
```

样例输出 Copy

```
3 3
```

```
#include <iostream>
using namespace std;
const int N = 105;

int n, m;
int p[N], cnt[N];

inline int find(int x)
{
    if(x != p[x]) p[x] = find(p[x]);
    return p[x];
}

int main()
{
    scanf("%d%d", &n, &m);
    for(int i = 1; i <= n; ++ i) p[i] = i, cnt[i] = 1;
    int res = 1;
    while(m -- )
    {
        int a, b;
        scanf("%d%d", &a, &b);
        int pa = find(a), pb = find(b);
        if(pa == pb) continue;

        cnt[pb] += cnt[pa];
        res = max(res, cnt[pb]);
        p[pa] = pb;
    }
    int a = 0;
    for(int i = 1; i <= n; ++ i)
        if(p[i] == i)
            a ++;
    printf("%d %d\n", a, res);
    return 0;
}
```

```python
list1 = input().split()
m = eval(list1[0])
t = eval(list1[1])
set_list = []
num_list = list(range(1, m + 1))
for i in range(t):
    list2 = input().split()
    a = eval(list2[0])
    b = eval(list2[1])
    flag = True
    for a_set in set_list:
        if a in a_set or b in a_set:
            a_set.add(a)
            a_set.add(b)
            flag = False
            break
    if flag is True:
        set_list.append({a, b})
max_p = 1
for a_set in set_list:
    for p in a_set:
        num_list.remove(p)
    if len(a_set) > max_p:
        max_p = len(a_set)
print("{} {}".format(len(set_list) + len(num_list), max_p))
/*************************************************************
```

问题 G: 整理试卷

时间限制: 1 Sec 内存限制: 128 MB

题目描述

学期末，老师们将成绩录入教务系统以后，将试卷装订后送到档案室，按照专业班级分成了不同的堆。教务赵老师决定把所有试卷合成一堆。

每一次合并，赵老师可以把两堆试卷合并到一起，消耗的体力等于两堆试卷的重量之和。可以看出，所有的试卷经过 m-1 次合并之后，就只剩下一堆了。赵老师在合并试卷时总共消耗的体力等于每次合并所耗体力之和。

因为还要花大力气把这些试卷搬到档案架上，所以赵老师在合并试卷时要尽可能地节省体力。假定每个试卷重量都为 1，并且已知试卷的种类数和每种试卷的数目，你的任务是设计出合并的次序方案，使赵老师耗费的体力最少，并输出这个最小的体力耗费值。

例如有 3 种试卷，数目依次为 1，2，9。可以先将 1、2 堆合并，新堆数目为 3，耗费体力为 3。接着，将新堆与原先的第三堆合并，又得到新的堆，数目为 12，耗费体力为 12。所以赵老师总共耗费体力=3+12=15。可以证明 15 为最小的体力耗费值。

输入

包括两行，第一行是一个整数 m（1 <= m <= 30000），表示试卷的种类数。第二行包含 n 个整数，用空格分隔，第 j 个整数 b_i（1 <= b_j <= 20000）是第 j 种试卷的数目。

输出

包括一行，这一行只包含一个整数，也就是最小的体力耗费值。输入数据保证这个值小于 2^{31}。

【数据规模】

对于 30%的数据，保证有 m <= 100；

对于 50%的数据，保证有 m <= 5000；

对于全部的数据，保证有 m <= 30000。

样例输入 Copy

```
 3

1  2  9
```

样例输出 Copy

```
15
```

```java
import java.util.Arrays;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }
        int sum = 0;
        Arrays.sort(arr);
        for(int i = 0;i < n-1;i++){
            int e = arr[i] + arr[i+1];
            arr[i] = 0;
            arr[i+1] = e;
            sum += e;
            Arrays.sort(arr);
        }
        System.out.println(sum);
    }
}
```

```python
m = eval(input())
num_list = input().split()
new_list = []
total = 0
for num in num_list:
    new_list.append(eval(num))
new_list.sort()
for i in range(1, len(new_list)):
    total = total + new_list[i] + new_list[i - 1]
    new_list[i] += new_list[i - 1]
    new_list.sort()
print(total, end="")
```

```
#include<bits/stdc++.h>
using namespace std;
int main()
{
    int n,n2,a1[10010],a2[10010],sum=0;
    cin>>n;
    memset(a1,127,sizeof(a1));
    memset(a2,127,sizeof(a2));
    for(int i=0;i<n;i++)
    cin>>a1[i];
    sort(a1,a1+n);
    int i=0,j=0,k,w;
    for(k=1;k<n;k++)
    {
        w=a1[i]<a2[j]?a1[i++]:a2[j++];
        w+=a1[i]<a2[j]?a1[i++]:a2[j++];
        a2[n2++]=w;
        sum+=w;

    }

     cout<<sum;
}
```